

LAPORAN PRAKTIKUM
PRAKTIKUM PENGUJIAN PERANGKAT LUNAK
PERTEMUAN 8
AUTOMATION TESTING: WEB ELEMENT LOCATORS



Disusun oleh:

Nama : Januarsyah Akbar
NIM : 24/535846/SV/24314
Kelas : PL4B2
Dosen Pengampu : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
April 29, 2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Hasil dan Pembahasan	4
2.1 Tugas	4
2.1.1 Pengujian Login SauceDemo Sederhana (SauceDemoLoginTest.java)	4
Kesimpulan	9

Tujuan Praktikum

1. Mahasiswa mampu memahami berbagai teknik lokator elemen web.
2. Mahasiswa mampu mengimplementasikan pencarian elemen menggunakan berbagai tipe locators seperti Name, XPath, dan Tag Name pada pengujian antarmuka dengan Selenium WebDriver.

Hasil dan Pembahasan

2.1 Tugas

Berikut adalah penjelasan dan tahap praktikum untuk pembuatan dan eksekusi *Automated Testing* terhadap interaksi aplikasi spesifik, yakni portal SauceDemo menggunakan berbagai tipe *Web Element Locators*. Pengujian dilakukan berdasarkan skenario *test case login* dengan langkah-langkah sebagai berikut:

1. Mengecek keberadaan teks “Swag Labs”.
2. Mengecek keberadaan *field username*.
3. Membersihkan (*clear*) *field username*.
4. Mengisi *field username*.
5. Mengecek keberadaan *field password*.
6. Membersihkan (*clear*) *field password*.
7. Mengisi *field password*.
8. Mengecek keberadaan tombol *login*.
9. Melakukan klik pada tombol *login*.

2.1.1 Pengujian Login SauceDemo Sederhana (SauceDemoLoginTest.java)

Pada bagian ini, kita akan membuat kelas `SauceDemoLoginTest` untuk memverifikasi proses *login* dengan mencari elemen-elemennya menggunakan locators spesifik seperti Name, XPath, dan Tag Name.

1. Mendefinisikan Package dan Import

Langkah pertama adalah melakukan *import library* yang dibutuhkan dari Selenium WebDriver dan JUnit 5:

```
1 import org.junit.jupiter.api.AfterEach;  
2 import org.junit.jupiter.api.BeforeEach;  
3 import org.junit.jupiter.api.Test;  
4 import org.openqa.selenium.By;  
5 import org.openqa.selenium.WebDriver;  
6 import org.openqa.selenium.WebElement;
```

```

7 import org.openqa.selenium.safari.SafariDriver;
8
9 import static org.junit.jupiter.api.Assertions.*;

```

Listing .1: Import Dependencies

2. Persiapan WebDriver (Setup)

Selanjutnya, kita mendefinisikan kelas pengujian dan melakukan persiapan awal menggunakan anotasi `@BeforeEach`. Inisialisasi *browser* menggunakan `SafariDriver`.

```

1 public class SauceDemoLoginTest {
2
3     private WebDriver driver;
4
5     @BeforeEach
6     public void setUp() {
7         driver = new SafariDriver();
8     }

```

Listing .2: Inisialisasi Kelas dan Setup WebDriver

3. Helper Methods untuk Pencarian Elemen

Untuk menghindari repetisi serta *error* yang tak tertangani, dibuat helper method `find` untuk menyederhanakan pencarian elemen dan `isPresent` untuk mengecek apakah elemen muncul.

```

1     private WebElement find(By locator) {
2         return driver.findElement(locator);
3     }
4
5     private boolean isPresent(By locator) {
6         try {
7             return !driver.findElements(locator).isEmpty();
8         } catch (Exception e) {
9             return false;
10        }
11    }

```

Listing .3: Helper Method

4. Skenario Pengujian Login dengan Beragam Locator

Skenario utama mengarahkan web driver ke laman SauceDemo dan menggunakan locators untuk melakukan interaksi. `By.xpath` digunakan untuk validasi teks header. `By.name` dipakai untuk input username. Sedangkan password ditemukan menggunakan *XPath axes (following)*. Terakhir, `By.tagName` diiterasikan untuk mencari tombol berjenis submit.

```

1     @Test
2     public void testLogin() {
3         driver.get("https://www.saucedemo.com/");
4
5         // 1) cek text "Swag Labs" ada menggunakan XPath (text node)
6         By swagText = By.xpath("//*[@text()='Swag Labs']");
7         assertTrue(isPresent(swagText), "Swag Labs text should be present");

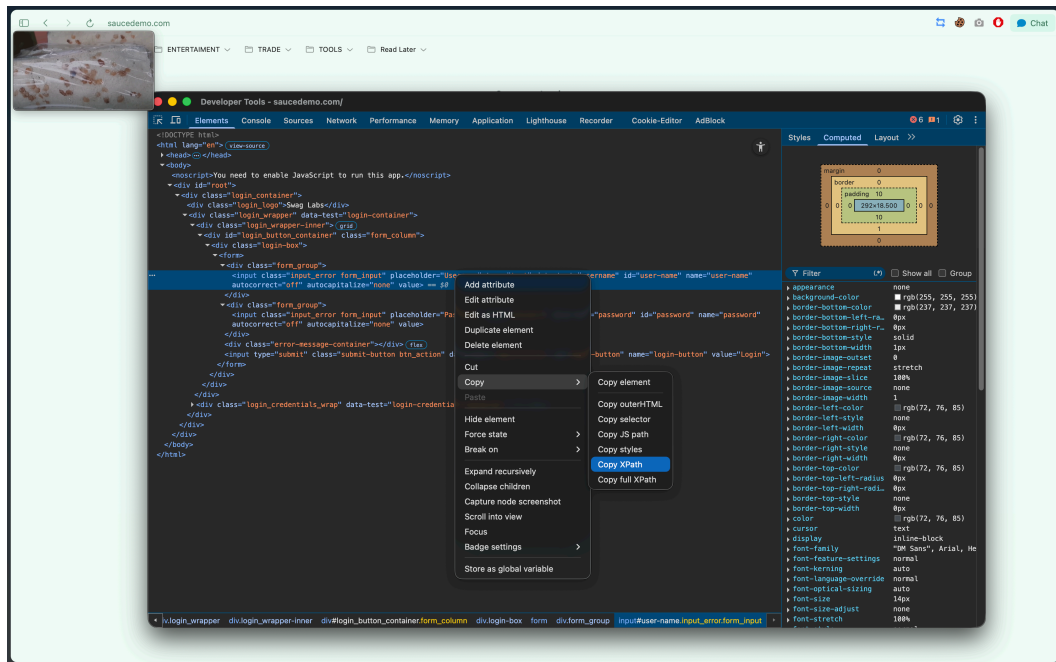
```

```

8
9 // 2) cek field username ada (gunakan locator by NAME)
10 By usernameBy = By.name("user-name");
11 assertTrue(isPresent(usernameBy), "Username field should be present");
12
13 WebElement usernameInput = find(usernameBy);
14 usernameInput.clear();
15 usernameInput.sendKeys("standard_user");
16
17 // 3) cek field password menggunakan XPath dengan axes
18 By passwordBy = By.xpath("//input[@name='user-name']/following::input[@type
19 ↪ = 'password'][1]");
20 assertTrue(isPresent(passwordBy), "Password field should be present");
21
22 WebElement passwordInput = find(passwordBy);
23 passwordInput.clear();
24 passwordInput.sendKeys("secret_sauce");
25
26 // 4) cek tombol login menggunakan TAG / collection approach
27 java.util.List<WebElement> inputs = driver.findElements(By.tagName("input"))
28 ↪ ;
29 WebElement loginButton = null;
30 for (WebElement el : inputs) {
31     String type = el.getAttribute("type");
32     if (type != null && (type.equalsIgnoreCase("submit") || type.
33 ↪ equalsIgnoreCase("button"))) {
34         loginButton = el;
35         break;
36     }
37 }
38
39 assertNotNull(loginButton, "Login button should be present");
40
41 loginButton.click();
42
43 String currentUrl = driver.getCurrentUrl();
44 assertEquals("https://www.saucedemo.com/inventory.html", currentUrl);
45 }

```

Listing .4: Test Method Login Locator



Gambar 1: Penerapan Locators Menggunakan XPath, Name, dan TagName

5. Membersihkan WebDriver (Teardown)

Bagian terakhir menggunakan anotasi `@AfterEach` ditujukan untuk mengakhiri sesi *browser* secara aman:

```

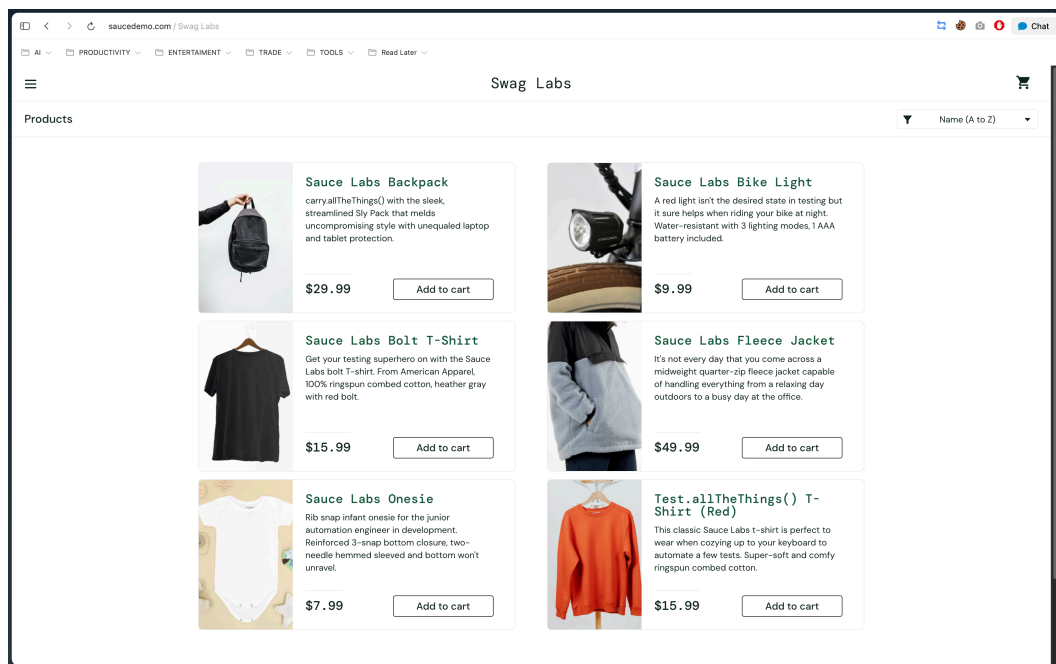
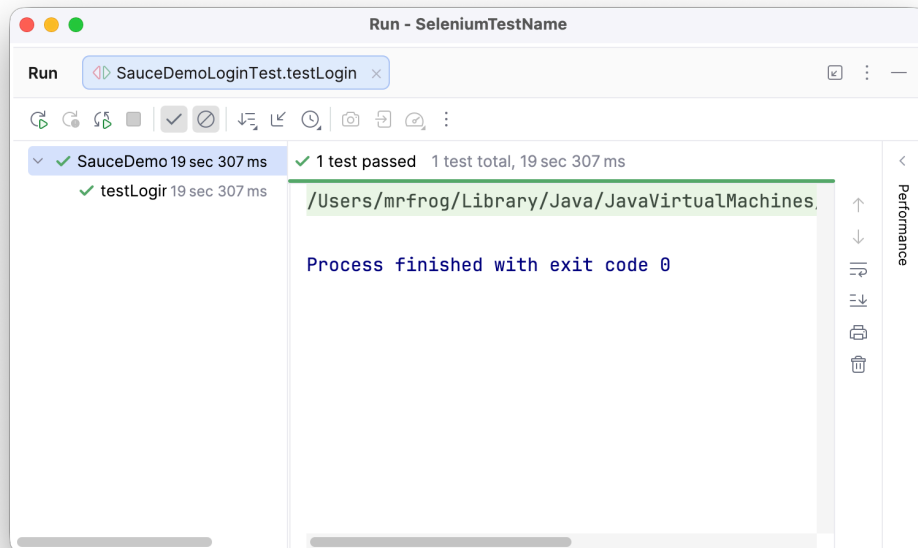
1  @AfterEach
2  public void tearDown() {
3      if (driver != null) {
4          driver.quit();
5      }
6  }
7  }

```

Listing .5: Penutupan Sesi Browser

Hasil Test

Berikut adalah tampilan interaksi sukses yang mengkonfirmasi *locators* berhasil menjangkau komponen front-end aplikasi web tersebut.



Gambar 2: Hasil Eksekusi Test dengan Implementasi Locators

Repository

akses kode lengkap untuk latihan dan tugas praktikum ini dapat ditemukan pada repository GitHub berikut:

GitHub Repository

Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan *kakas bantu* seperti Selenium WebDriver sangat mempermudah pembuatan *automated testing* atau pengujian otomatis antarmuka pada suatu aplikasi secara terstruktur. Pengujian interaksi pada formulir *login* di website **SauceDemo** dengan mengimplementasikan berbagai *Web Element Locators* (seperti Name, XPath, dan Tag Name) telah berhasil diotomatisasi secara terperinci.

Selain itu, pendokumentasian skrip pengujian berbasis *Java* serta JUnit telah menunjukkan bagaimana *webdriver* diinstansiasi, dinavigasi mencari *element* dan divalidasi dengan struktur kode secara langsung. Ini sangat menghemat waktu dibandingkan dengan menguji suatu *form login* dan interaksi-interaksi web secara manual berulang-ulang, menstimulasi produktivitas untuk QA (Quality Assurance) serta *Developer* pada tahap-tahap integrasi perangkat lunak nantinya.